



RETAIL STORE SALES ANALYSIS USING R PROGRAMMING

PROJECT
REPORT

Table of Contents

1. Introduction.....	4
2. Objectives of the Project.....	4
3. Dataset Description.....	4
4. Methodology	5
5. Data Analysis.....	5
◆ <code>setwd("C:/Users/PC/Desktop/R project")</code>	6
◆ <code>file.create("retail_sales_project.R")</code>	6
◆ <code>sales <- read.csv("retail_store_sales.csv")</code>	6
◆ <code>str(sales)</code>	8
◆ <code>summary(sales)</code>	8
◆ <code>head(sales)</code>	8
6.3 VECTORS & DATA TYPES.....	9
6.4 VECTOR ARITHMETIC & INDEXING.....	9
6.5 Overview of Sales Data by using LIST.....	10
What is sales_list?	11
6.8 DATA FRAME OPERATIONS.....	13
6.12 Category-wise Analysis	19
6.13 Product-wise Analysis	20
6.14 Location / Store-wise Analysis	20
7. VISUALIZATION.....	21
7.1 Bar Chart (Category vs Total Sales)	21
Interpretation of Category-wise Total Sales.....	22
7.2 Bar Chart (Top 10 Items by Sales)	23
Interpretation of Top 10 Items by Sales.....	25
7.3 Bar Chart (Sales by Location)	25
Interpretation of Sales by Location Chart.....	27
7.4 Scatter plot (Price.Per.Unit by Quantity)	27
What does the graph show?	28
★ Easy Findings	28
7.5 Line Graph (Category vs Total Spent)	29
8. Clean data (tidying strings).....	31

8.1 Tidying Strings by <code>str_trim()</code> > <code>str_to_title()</code>	31
9. Filtering Data	32
9.1 Filter or subset data (<code>dplyr</code>).....	32
10. Grouping and Summarizing Data	33
10.1 Find top customer spending	33
10.2 Check discounts applied.....	34
11. Removing Duplicate transactions	35
11.1 Clean duplicates (distinct transactions).....	35
What does this code do?	35
STEP 1: Package name	37
STEP 2: Package folder creation	37
STEP 3: Basic structure	37
STEP 4: DESCRIPTION file	37
STEP 5: Function 1 – Total Sales	37
STEP 6: Function 2 – Category-wise Sales	38
STEP 7: Function 3 – Top Selling Items	38
STEP 8: Function 4 – Average Order Value	38
STEP 9: Function 5 – Discount Analysis	39
STEP 10: NAMESPACE file update	40
STEP 11: Package load	40
STEP 12: Using in my project	41

1. Introduction

In today's digital world, data plays a vital role in business decision-making. Retail businesses generate large amounts of sales data every day. Analyzing this data helps organizations understand customer behavior, product performance, and sales trends.

This project focuses on analyzing **Retail Store Sales Data using R Programming**. The aim is to clean, process, analyze, and visualize sales data to extract meaningful insights. R is used because it is powerful for data manipulation, statistical analysis, and visualization.

2. Objectives of the Project

The main objectives of this project are:

- To import and explore retail sales data in R
- To clean missing and inconsistent data
- To analyze sales by **Category, Item, Location, Customer, and Discounts**

To identify:

- Best selling categories
- Top selling products
- Most profitable locations
- High-value customers
- To visualize sales patterns using graph
- To support better business decisions using data

3. Dataset Description

The dataset contains **12,575 retail transactions** with 11 variables:

- Transaction.ID
- Customer.ID
- Category
- Item
- Price.Per.Unit
- Quantity
- Total.Spent
- Payment.Method
- Location
- Transaction.Date
- Discount.Applied

4. Methodology

The project followed these steps:

1. Import data into R
2. Explore structure and summary
3. Handle missing values
4. Clean and standardize text
5. Create subsets
6. Perform category-wise, item-wise, location-wise analysis
7. Create visualizations
8. Interpret results

5. Data Analysis

The following analyses were performed:

- ✓ Category-wise Total Sales
- ✓ Item-wise Total Sales
- ✓ Location-wise Sales
- ✓ Customer Spending Analysis
- ✓ Discount Impact Analysis

6. Code and Analysis

6.1 Setting up working directory and reading CSV File in R Studio

```
> setwd("C:/Users/PC/Desktop/R project")
> file.create("retail_sales_project.R")
[1] TRUE
> # Import data
> sales <- read.csv("retail_store_sales.csv")
```

◆ `setwd("C:/Users/PC/Desktop/R project")`

This sets the **working directory**.

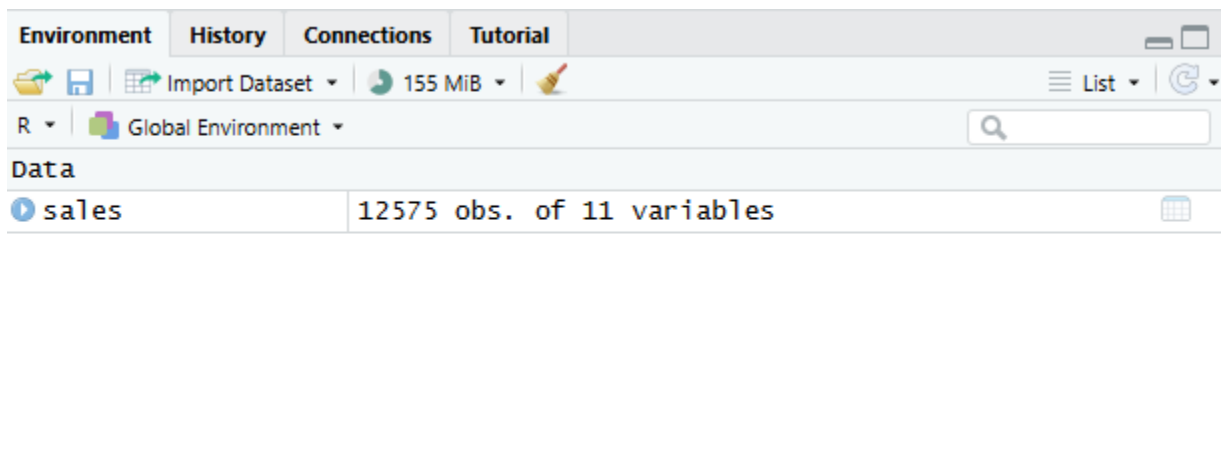
It tells R where your project files are stored, so it can easily read and save files.

◆ `file.create("retail_sales_project.R")`

This creates a new **R script file** for your project where you write all your code.

◆ `sales <- read.csv("retail_store_sales.csv")`

This imports the CSV file into R and stores it in a variable called `sales`.



6.2 Inspecting data using str(), summary(), head()

```
# Inspect data
```

```
str(sales)
```

```
summary(sales)
```

```
head(sales)
```

```
> # Inspect data
> str(sales)
'data.frame': 12575 obs. of 11 variables:
 $ Transaction.ID : chr "TXN_6867343" "TXN_3731986" "TXN_9303719" "TXN_9458126" ...
 $ Customer.ID : chr "CUST_09" "CUST_22" "CUST_02" "CUST_06" ...
 $ Category : chr "Patisserie" "Milk Products" "Butchers" "Beverages" ...
 $ Item : chr "Item_10_PAT" "Item_17_MILK" "Item_12_BUT" "Item_16_BEV" ...
 $ Price.Per.Unit : num 18.5 29 21.5 27.5 12.5 NA 5 33.5 27.5 36.5 ...
 $ Quantity : int 10 9 2 9 7 10 8 NA 1 3 ...
 $ Total.Spent : num 185 261 43 247.5 87.5 ...
 $ Payment.Method : chr "Digital wallet" "Digital wallet" "Credit Card" "Credit Card"
 ...
 $ Location : chr "Online" "online" "online" "online" ...
 $ Transaction.Date: chr "08/04/2024" "23/07/2023" "05/10/2022" "07/05/2022"
```

```
> summary(sales)
Transaction.ID      Customer.ID      Category      Item
Length:12575      Length:12575      Length:12575      Length:12575
Class :character    Class :character    Class :character    Class :character
Mode :character      Mode :character      Mode :character      Mode :character
```

Price.Per.Unit	Quantity	Total.Spent	Payment.Method
Min. : 5.00	Min. : 1.000	Min. : 5.0	Length:12575
1st Qu.:14.00	1st Qu.: 3.000	1st Qu.: 51.0	Class :character
Median :23.00	Median : 6.000	Median :108.5	Mode :character
Mean :23.37	Mean : 5.536	Mean :129.7	
3rd Qu.:33.50	3rd Qu.: 8.000	3rd Qu.:192.0	
Max. :41.00	Max. :10.000	Max. :410.0	
NA's :609	NA's :604	NA's :604	
Location	Transaction.Date	Discount.Applied	
Length:12575	Length:12575	Mode :logical	
Class :character	Class :character	FALSE:4157	
Mode :character	Mode :character	TRUE :4219	
		NA's :4199	


```
> head(sales)
  Transaction.ID Customer.ID      Category      Item Price.Per.Unit Quantity
1   TXN_6867343   CUST_09    Patisserie  Item_10_PAT         18.5         10
2   TXN_3731986   CUST_22 Milk Products  Item_17_MILK         29.0          9
3   TXN_9303719   CUST_02     Butchers  Item_12_BUT         21.5          2
4   TXN_9458126   CUST_06   Beverages  Item_16_BEV         27.5          9
5   TXN_4575373   CUST_05         Food  Item_6_FOOD         12.5          7
6   TXN_7482416   CUST_09    Patisserie         NA         10
  Total.Spent Payment.Method Location Transaction.Date Discount.Applied
1      185.0 Digital wallet   Online      08/04/2024          TRUE
2      261.0 Digital wallet   Online      23/07/2023          TRUE
3       43.0  Credit Card   Online      05/10/2022         FALSE
4      247.5  Credit Card   Online      07/05/2022          NA
5       87.5 Digital wallet   Online      02/10/2022         FALSE
6      200.0  Credit Card   Online      30/11/2023          NA
> |
```

◆ `str(sales)`

This shows the **structure of the dataset**:

- Number of rows and columns
- Data types of each variable

✦ We use it to understand what kind of data we are working with.

◆ `summary(sales)`

This gives **summary statistics**:

- Min, Max, Mean, Median
- Count of missing values (NA)

✦ We use it to get an overall idea of the data quality and distribution.

◆ `head(sales)`

This shows the **first 6 rows** of the dataset.

✦ We use it to quickly see what the data looks like

6.3 VECTORS & DATA TYPES

```
# Creating vectors
prices <- c(18.5, 29, 21.5, 27.5)
categories <- c("Food", "Beverages", "Milk")
discount <- c(TRUE, FALSE, TRUE)

# Data types check
class(prices)
class(categories)
class(discount)
```

Data	
 sales	12575 obs. of 11 variables 
values	
categories	chr [1:3] "Food" "Beverages" "Milk"
discount	logi [1:3] TRUE FALSE TRUE
prices	num [1:4] 18.5 29 21.5 27.5

```
> # Data types check
> class(prices)
[1] "numeric"
> class(categories)
[1] "character"
> class(discount)
[1] "logical"
>
```

6.4 VECTOR ARITHMETIC & INDEXING

```
# Indexing
prices[1]
categories[2]

# Vector arithmetic
prices + 5
prices * 2
```

```

> # Indexing
> prices[1]
[1] 18.5
> categories[2]
[1] "Beverages"

> # Vector arithmetic
> prices + 5
[1] 23.5 34.0 26.5 32.5
> prices * 2
[1] 37 58 43 55

```

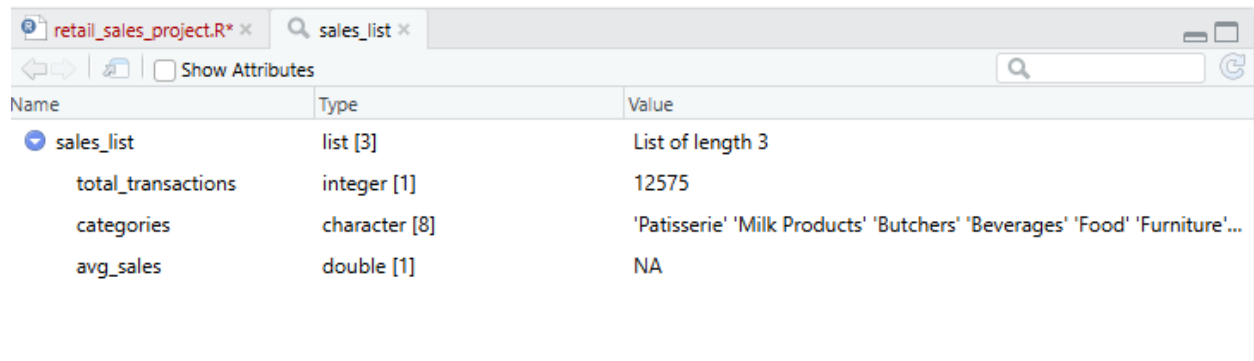
6.5 Overview of Sales Data by using LIST

```

# List
sales_list <- list(
  total_transactions = nrow(sales),
  categories = unique(sales$Category),
  avg_sales = mean(sales$Total.Spent)
)

sales_list

```



Name	Type	Value
sales_list	list [3]	List of length 3
total_transactions	integer [1]	12575
categories	character [8]	'Patisserie' 'Milk Products' 'Butchers' 'Beverages' 'Food' 'Furniture'...
avg_sales	double [1]	NA

Note: our total spent column contains NA values that's why the average sales are NA

We are **creating a list** in R. A list is like a **container** that can hold **different types of information together** – numbers, text, vectors, even other lists.

Here's what each part does:

- `total_transactions = nrow(sales)` → Counts how many rows are in your `sales` dataset.
 - Think of it as **how many sales records** you have.
- `categories = unique(sales$Category)` → Finds all the **different categories** in your sales dataset.
 - Example: "Clothing", "Electronics", "Toys"
- `avg_sales = mean(sales$Total.Spent)` → Calculates the **average amount spent** in all transactions.

What is `sales_list`?

When you run `sales_list`, it will **show all three things together** in a neat list.

- It is **like a small report** about your sales: total transactions, categories, and average spending.

6.6 Removing the NA values from total spent column

```
sales_list <- list(  
  total_transactions = nrow(sales),  
  categories = unique(sales$Category),  
  avg_sales = mean(sales$Total.Spent, na.rm = TRUE)  
)  
sales_list
```

```
> sales_list  
$total_transactions  
[1] 12575  
  
$categories  
[1] "Patisserie"           "Milk Products"  
[3] "Butchers"             "Beverages"  
[5] "Food"                 "Furniture"  
[7] "Electric household essentials" "Computers and electric accessories"  
  
$avg_sales  
[1] 129.6526  
  
> view(sales_list)
```

Name	Type	Value
sales_list	list [3]	List of length 3
total_transactions	integer [1]	12575
categories	character [8]	'Patisserie' 'Milk Products' 'Butchers' 'Beverages' 'Food' 'Furniture'...
avg_sales	double [1]	129.6526

6.7 Saving new CSV File without NA values

```
sales_clean <- sales
sales_clean <- sales_clean[!is.na(sales_clean$Total.Spent), ]
write.csv(
  sales_clean,
  "retail_store_sales_cleaned.csv",
  row.names = FALSE
)
```

6.8 DATA FRAME OPERATIONS

```
sales_clean <- read.csv("retail_store_sales_cleaned.csv")
```

```
# Access columns
sales_clean$Category
sales_clean$Total.Spent
```

```
> sales$Category
 [1] "Patisserie"      "Milk Products"
 [3] "Butchers"        "Beverages"
 [5] "Food"            "Patisserie"
 [7] "Food"            "Furniture"
 [9] "Furniture"       "Butchers"
[11] "Butchers"        "Milk Products"
[13] "Food"            "Patisserie"
[15] "Milk Products"   "Beverages"
[17] "Patisserie"      "Milk Products"
[19] "Electric household essentials" "Furniture"
[21] "Beverages"       "Milk Products"
[23] "Electric household essentials" "Food"
[25] "Furniture"       "Furniture"
[27] "Butchers"        "Furniture"
[29] "Food"            "Patisserie"
[31] "Food"            "Patisserie"
[33] "Food"            "Electric household essentials"
[35] "Patisserie"      "Beverages"
[37] "Food"            "Furniture"
[39] "Beverages"       "Butchers"
[41] "Beverages"       "Electric household essentials"
[43] "Butchers"        "Beverages"
[45] "Food"            "Food"
[47] "Computers and electric accessories" "Patisserie"
[49] "Butchers"        "Milk Products"
[51] "Butchers"        "Computers and electric accessories"
[53] "Food"            "Milk Products"
[55] "Furniture"       "Furniture"
[57] "Computers and electric accessories" "Computers and electric accessories"
[59] "Furniture"       "Milk Products"
[61] "Computers and electric accessories" "Milk Products"
[63] "Patisserie"      "Computers and electric accessories"
[65] "Food"            "Butchers"
[67] "Patisserie"      "Patisserie"
```

```

> sales_clean$Total.Spent
 [1] 185.0 261.0  43.0 247.5  87.5 200.0  40.0  27.5 109.5  72.0  52.0  45.5 237.0
[14]  55.0 232.0 275.0  23.0 126.0 105.0  66.5  18.5  49.0 134.0 410.0 245.0 182.5
[27] 200.0 100.0 196.0 315.0 287.0  76.0  92.5 100.0 190.0 255.5  92.5 276.5  46.5
[40]   8.0 107.5 165.0  80.0 165.0 192.5 335.0  66.0 215.0  96.0 190.0  42.0  11.0
[53] 234.0 247.5  87.5 316.0  82.5 180.0 255.5 365.0  39.0 245.0 172.0 134.0 122.0
[66]  30.0  66.5 365.0 126.0  84.0 320.0 219.0 335.0  67.0 146.0 290.0  70.0 160.0
[79]  82.0  14.0 105.0 355.5 124.0  28.5  49.0  47.5 193.5  38.0  14.0  76.0  12.5
[92] 140.0 120.0 183.0 335.0 165.0 305.0  41.0 155.0  92.5  19.5 287.0  33.0 108.5
[105] 109.5 119.0 280.0 155.0 287.0  62.0  32.0 380.0 304.0 139.5 114.0 192.0 167.5
[118]  92.5 275.0  88.0 395.0  70.0 158.0  25.0 410.0  50.0 395.0  32.5 122.0  15.0
[131]  33.5  24.0 111.0 120.0  40.0 395.0  46.0  25.0  32.5 305.0  36.5  62.5  25.0
[144] 280.0 139.5 219.0 161.0  26.0 126.0  98.0 126.0 315.0  85.0  41.0 126.0  84.0
[157] 287.0  98.0 192.0  80.0 228.0  91.5 140.0  93.0  55.0 335.0  35.0  82.0 180.0
[170]  88.0 192.5  40.0 167.5 261.0 208.0 100.5 167.5 275.0 134.0  38.0   9.5  42.0
[183] 247.5  10.0 138.0 288.0 167.5  60.0 123.0  43.0  73.0 197.5  69.0  61.0 140.0
[196]  52.0 116.0  70.0  46.5  35.0  50.0 123.0  76.0  40.0  47.5 245.0  55.5   6.5
[209]  82.0  93.0 152.5 118.5  35.0 123.0  25.0 172.0 280.0 190.0 192.5  74.0  66.0
[222] 105.0 205.0 150.5 110.0  10.0  19.0   6.5  48.0 175.0 369.0 255.5 150.5  37.5
[235] 150.5  65.0 170.0  77.5 129.5  15.5  24.5 118.5 147.0  37.5  21.5  46.5  77.5
[248]  72.0  55.0  35.0  98.0  31.0  38.0  12.5  99.0 170.0 123.0 128.0 234.0 234.0
[261]  58.5   6.5  48.0 129.5 237.0  55.0  66.0  50.0 266.0 138.0  29.0 167.5 288.0
[274] 158.0 268.0  50.0  64.0  15.5 128.0 115.0  32.0  39.0  80.0 119.0 276.5  80.0
[287]  69.0 355.5  93.0  64.5 260.0  55.0 232.0  41.0 207.0 224.0 335.0  52.0  64.0
[300]  73.0  35.0 170.0 120.0 207.0 185.0  80.0 232.0 328.0  92.0  93.0 120.0  60.0
[313] 244.0  33.0  39.5  37.5 237.0  15.0  67.0 410.0 328.0 100.0  69.0  43.0  23.0
[326] 268.0  16.0 328.5 129.5 147.0 118.5 182.0 118.5  61.0  44.0  73.0  11.0  76.0
[339] 234.0 129.0  92.0  72.0 301.5  80.0 255.5 182.0  99.0  57.0  43.0  29.0 115.0
[352]  98.0   5.0  98.0 369.0 108.5 320.0 120.0  70.0  38.0  96.0  52.0 122.5 261.0
[365] 220.0  35.0 261.0  15.5  67.0  23.0  24.0 112.5 275.0 190.0  68.0  64.0 182.5
[378]  60.0  60.0 222.0 216.0  72.0  68.0 280.0 220.0  22.5 228.0  64.5 172.0 147.0

```

6.9 Clearing the NA values from other columns

Before (Yellow colour highlights the boxes with the NA values)

 **sales** | 12575 obs. of 11 variables

	A	B	C	D	E	F	G	H	I	J	K
1	Transaction ID	Customer ID	Category	Item	Price Per Unit	Quantity	Total Spent	Payment Method	Location	Transaction Date	Discount Applied
2	TXN_6867343	CUST_09	Patisserie	Item_10_PAT	18.5	10	185	Digital Wallet	Online	8/4/2024	TRUE
3	TXN_3731986	CUST_22	Milk Products	Item_17_MILK	29	9	261	Digital Wallet	Online	23/07/2023	TRUE
4	TXN_9303719	CUST_02	Butchers	Item_12_BUT	21.5	2	43	Credit Card	Online	5/10/2022	FALSE
5	TXN_9458126	CUST_06	Beverages	Item_16_BEV	27.5	9	247.5	Credit Card	Online	7/5/2022	FALSE
6	TXN_4575373	CUST_05	Food	Item_6_FOOD	12.5	7	87.5	Digital Wallet	Online	2/10/2022	FALSE
7	TXN_74822416	CUST_09	Patisserie			10	200	Credit Card	Online	30/11/2023	
8	TXN_3652209	CUST_07	Food	Item_1_FOOD	5	8	40	Credit Card	In-store	10/6/2023	TRUE
9	TXN_1372952	CUST_21	Furniture		33.5			Digital Wallet	In-store	2/4/2024	TRUE
10	TXN_9728486	CUST_23	Furniture	Item_16_FUR	27.5	1	27.5	Credit Card	In-store	26/04/2023	FALSE
11	TXN_2722661	CUST_25	Butchers	Item_22_BUT	36.5	3	109.5	Cash	Online	14/03/2024	FALSE
12	TXN_8776416	CUST_22	Butchers	Item_3_BUT	8	9	72	Cash	In-store	14/12/2024	TRUE
13	TXN_5422631	CUST_09	Milk Products			8	52	Digital Wallet	In-store	12/1/2025	TRUE
14	TXN_5874772	CUST_23	Food	Item_2_FOOD	6.5	7	45.5	Cash	Online	9/9/2023	TRUE
15	TXN_4413070	CUST_14	Patisserie	Item_24_PAT	39.5	6	237	Digital Wallet	In-store	20/05/2022	FALSE
16	TXN_2490363	CUST_09	Milk Products	Item_16_MILK	27.5	2	55	Digital Wallet	Online	22/05/2022	
17	TXN_1809665	CUST_14	Beverages		24.5			Credit Card	In-store	11/5/2022	
18	TXN_7563311	CUST_23	Patisserie	Item_17_PAT	29	8	232	Cash	Online	16/11/2024	TRUE
19	TXN_9634894	CUST_15	Milk Products			10	275	Digital Wallet	Online	17/04/2022	
20	TXN_4396807	CUST_17	Electric household essentials	Item_13_EHE	23	1	23	Digital Wallet	In-store	7/2/2022	FALSE
21	TXN_4206593	CUST_01	Furniture		35			Digital Wallet	Online	13/01/2025	FALSE
22	TXN_9939063	CUST_14	Beverages	Item_7_BEV	14	9	126	Digital Wallet	In-store	14/01/2024	
23	TXN_8685338	CUST_15	Milk Products			3	105	Credit Card	In-store	29/10/2023	

retail_store_sales original

After:

```
# Read the CSV file
```

```
sales_clean <- read.csv("retail_store_sales_cleaned.csv",
stringsAsFactors = FALSE)
```

```
# Remove rows with any NA values
```

```
sales_clean <- na.omit(sales_clean)
```

```
# Overwrite the same CSV file
```

```
write.csv(sales_clean, "retail_store_sales_cleaned.csv",
row.names = FALSE)
```

 **sales_clean** | 7579 obs. of 11 variables

	A	B	C	D	E	F	G	H	I	J	K
1	Transactio	Customer.ID	Category	Item	Price.Per.Unit	Quantity	Total.Spent	Payment.Method	Location	Transaction.Date	Discount.Applied
2	TXN_6867	CUST_09	Patisserie	Item_10_PAT	18.5	10	185	Digital Wallet	Online	8/4/2024	TRUE
3	TXN_3731	CUST_22	Milk Products	Item_17_MILK	29	9	261	Digital Wallet	Online	23/07/2023	TRUE
4	TXN_9303	CUST_02	Butchers	Item_12_BUT	21.5	2	43	Credit Card	Online	5/10/2022	FALSE
5	TXN_4575	CUST_05	Food	Item_6_FOOD	12.5	7	87.5	Digital Wallet	Online	2/10/2022	FALSE
6	TXN_3652	CUST_07	Food	Item_1_FOOD	5	8	40	Credit Card	In-store	10/6/2023	TRUE
7	TXN_9728	CUST_23	Furniture	Item_16_FUR	27.5	1	27.5	Credit Card	In-store	26/04/2023	FALSE
8	TXN_2722	CUST_25	Butchers	Item_22_BUT	36.5	3	109.5	Cash	Online	14/03/2024	FALSE
9	TXN_8776	CUST_22	Butchers	Item_3_BUT	8	9	72	Cash	In-store	14/12/2024	TRUE
10	TXN_5874	CUST_23	Food	Item_2_FOOD	6.5	7	45.5	Cash	Online	9/9/2023	TRUE
11	TXN_4413	CUST_14	Patisserie	Item_24_PAT	39.5	6	237	Digital Wallet	In-store	20/05/2022	FALSE
12	TXN_7563	CUST_23	Patisserie	Item_17_PAT	29	8	232	Cash	Online	16/11/2024	TRUE
13	TXN_4396	CUST_17	Electric household essentials	Item_13_EHE	23	1	23	Digital Wallet	In-store	7/2/2022	FALSE
14	TXN_3314	CUST_04	Food	Item_10_FOOD	18.5	1	18.5	Cash	Online	30/09/2023	FALSE
15	TXN_9980	CUST_13	Furniture	Item_14_FUR	24.5	2	49	Cash	Online	29/08/2023	FALSE
16	TXN_9057	CUST_18	Butchers	Item_20_BUT	33.5	4	134	Credit Card	In-store	16/03/2022	TRUE
17	TXN_1599	CUST_14	Furniture	Item_25_FUR	41	10	410	Credit Card	In-store	24/03/2024	TRUE
18	TXN_1598	CUST_08	Food	Item_14_FOOD	24.5	10	245	Cash	In-store	23/10/2022	TRUE
19	TXN_7138	CUST_18	Patisserie	Item_22_PAT	36.5	5	182.5	Credit Card	In-store	10/3/2022	FALSE
20	TXN_5444	CUST_05	Food	Item_11_FOOD	20	10	200	Digital Wallet	Online	26/04/2022	FALSE
21	TXN_4353	CUST_22	Patisserie	Item_6_PAT	12.5	8	100	Credit Card	Online	5/4/2022	TRUE
22	TXN_6398	CUST_15	Beverages	Item_25_BEV	41	7	287	Credit Card	In-store	3/6/2024	TRUE
23	TXN_9065	CUST_02	Food	Item_23_FOOD	38	2	76	Digital Wallet	In-store	10/3/2024	FALSE
24	TXN_6867	CUST_09	Patisserie	Item_10_PAT	18.5	10	185	Digital Wallet	Online	8/4/2024	TRUE

6.10 Removing Outliers from Data

Step 1: Read your CSV

```
outliers <- read.csv("retail_store_sales_cleaned.csv")
```

Step 2: Identify numeric columns

```
num_cols <- sapply(outliers, is.numeric)
```

Step 3: Remove outliers using IQR method for all numeric columns

```
for(col in names(outliers)[num_cols]) {
  Q1 <- quantile(outliers[[col]], 0.25, na.rm = TRUE)
  Q3 <- quantile(outliers[[col]], 0.75, na.rm = TRUE)
  IQR <- Q3 - Q1
  lower <- Q1 - 1.5 * IQR
  upper <- Q3 + 1.5 * IQR
  # Keep only rows within this range
  outliers <- outliers[outliers[[col]] >= lower & outliers[[col]] <=
upper,]
```

```
# Overwrite the same CSV file

write.csv(outliers, "retail_store_sales_cleaned.csv", row.names =
FALSE)
```

Initial observations:

sales	12575 obs. of 11 variables
-------	----------------------------

After removing missing values:

sales_clean	7579 obs. of 11 variables
-------------	---------------------------

After removing the outliers:

outliers	7542 obs. of 11 variables
----------	---------------------------

6.11 Creating Subsets of each unique category

```
#Creating unique categories subsets

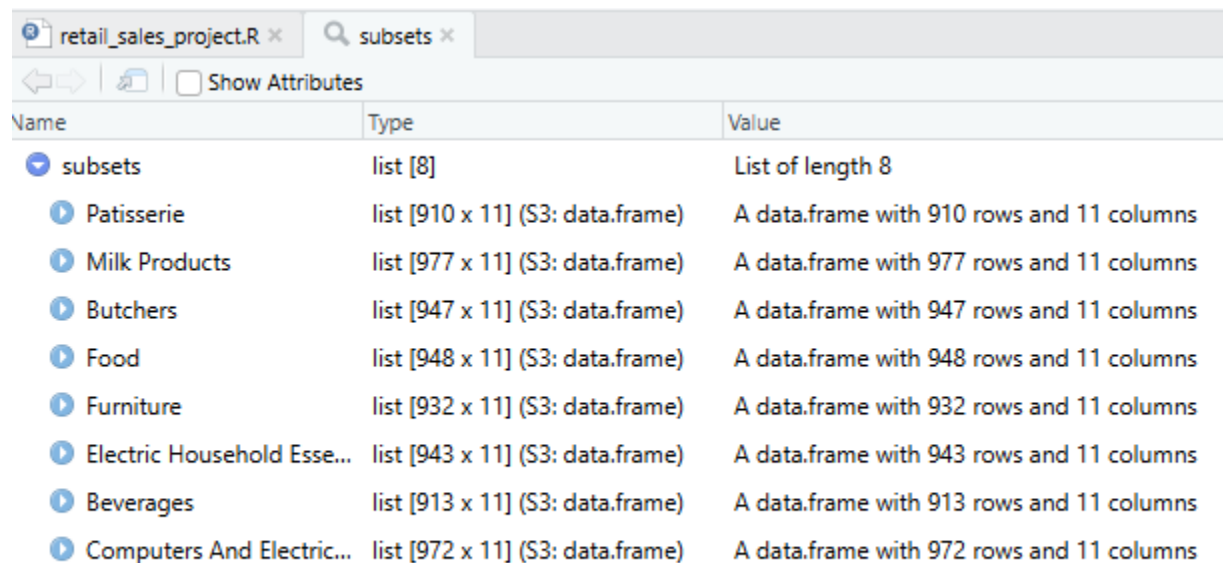
data <- read.csv("retail_store_sales_cleaned.csv")

unique_categories <- unique(data$Category)

unique_categories

> unique_categories <- unique(data$Category)
> unique_categories
[1] "Patisserie"           "Milk Products"
[3] "Butchers"            "Food"
[5] "Furniture"           "Electric household essentials"
[7] "Beverages"           "Computers and electric accessories"
```

```
#Creating subsets
subsets <- list()
for (cat in unique_categories) {
  subsets[[cat]] <- subset(data, Category == cat)
}
```



Name	Type	Value
subsets	list [8]	List of length 8
Patisserie	list [910 x 11] (S3: data.frame)	A data.frame with 910 rows and 11 columns
Milk Products	list [977 x 11] (S3: data.frame)	A data.frame with 977 rows and 11 columns
Butchers	list [947 x 11] (S3: data.frame)	A data.frame with 947 rows and 11 columns
Food	list [948 x 11] (S3: data.frame)	A data.frame with 948 rows and 11 columns
Furniture	list [932 x 11] (S3: data.frame)	A data.frame with 932 rows and 11 columns
Electric Household Esse...	list [943 x 11] (S3: data.frame)	A data.frame with 943 rows and 11 columns
Beverages	list [913 x 11] (S3: data.frame)	A data.frame with 913 rows and 11 columns
Computers And Electric...	list [972 x 11] (S3: data.frame)	A data.frame with 972 rows and 11 columns

Subsets show that out of 7542 total items of the retail store we have

- 910 patisserie sold
- 977 milk products sold
- 947 butchers sold
- 948 food items sold
- 932 furniture sold
- 943 electric household essentials sold
- 913 beverages sold
- 972 computer and electronics sold

Note that milk products are being sold the most, but we haven't said that it's generating maximum revenue or sales is highest in this category.

	Transaction.ID	Customer.ID	Category	Item	Price.Per.Unit	Quantity	Total.Spent	Payment.Me
2	TXN_3731986	CUST_22	Milk Products	Item_17_MILK	29.0	9	261.0	Digital We
36	TXN_3424571	CUST_09	Milk Products	Item_6_MILK	12.5	7	87.5	Credit Car
38	TXN_1998771	CUST_25	Milk Products	Item_16_MILK	27.5	3	82.5	Digital We
58	TXN_5705131	CUST_19	Milk Products	Item_4_MILK	9.5	4	38.0	Digital We
80	TXN_6989569	CUST_11	Milk Products	Item_21_MILK	35.0	2	70.0	Credit Car
83	TXN_3831408	CUST_18	Milk Products	Item_1_MILK	5.0	10	50.0	Digital We
86	TXN_5111184	CUST_19	Milk Products	Item_11_MILK	20.0	6	120.0	Credit Car
105	TXN_4448047	CUST_03	Milk Products	Item_8_MILK	15.5	6	93.0	Credit Car
108	TXN_8959624	CUST_09	Milk Products	Item_1_MILK	5.0	7	35.0	Digital We

6.12 Category-wise Analysis

☞ To analyze the performance of each category

- Which category has the highest sales
- Which category is weak

```
aggregate(Total.Spent ~ Category, data, sum)
```

```
> aggregate(Total.Spent ~ Category, data, sum)
      Category Total.Spent
1    Beverages    121643.0
2    Butchers    132046.5
3 Computers And Electric Accessories 126211.5
4   Electric Household Essentials    121921.5
5         Food    120794.5
6      Furniture    118296.5
7    Milk Products    115718.5
8    Patisserie    116711.0
```

6.13 Product-wise Analysis

☞ To identify Best & worst products.

- Top selling products
- Least selling products

```
aggregate(Total.Spent ~ Item, data, sum)
```

```
> aggregate(Total.Spent ~ Item, data, sum)
      Item Total.Spent
1  Item_1_BEV      565.0
2  Item_1_BUT      635.0
3  Item_1_CEA      420.0
4  Item_1_EHE      755.0
5  Item_1_FOOD      475.0
6  Item_1_FUR     1060.0
7  Item_1_MILK     1930.0
8  Item_1_PAT     1245.0
9  Item_10_BEV     2183.0
10 Item_10_BUT     1165.5
11 Item_10_CEA     5309.5
12 Item_10_EHE     2627.0
13 Item_10_FOOD     4033.0
14 Item_10_FUR     2479.0
15 Item_10_MILK       629.0
16 Item_10_PAT     1739.0
17 Item_11_BEV     4320.0
18 Item_11_BUT     2280.0
19 Item_11_CEA     5060.0
20 Item_11_EHE     3020.0
```

6.14 Location / Store-wise Analysis

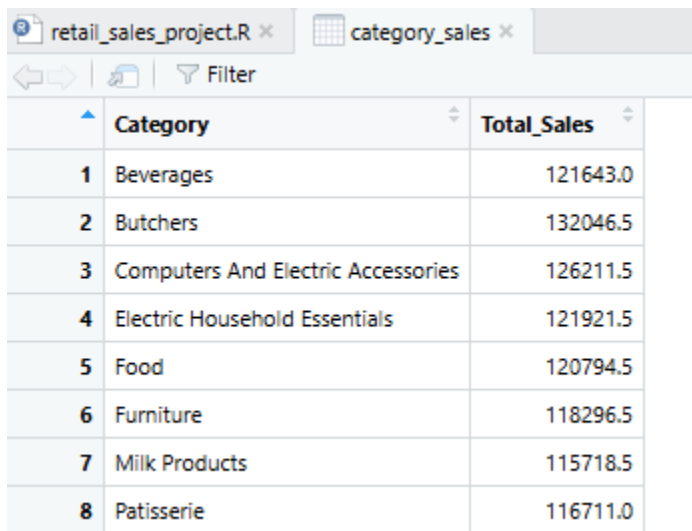
```
aggregate(Total.Spent ~ Location, data, sum)
```

```
      Location Total.Spent
1 In-store      485675
2  online      502838
```

7. VISUALIZATION

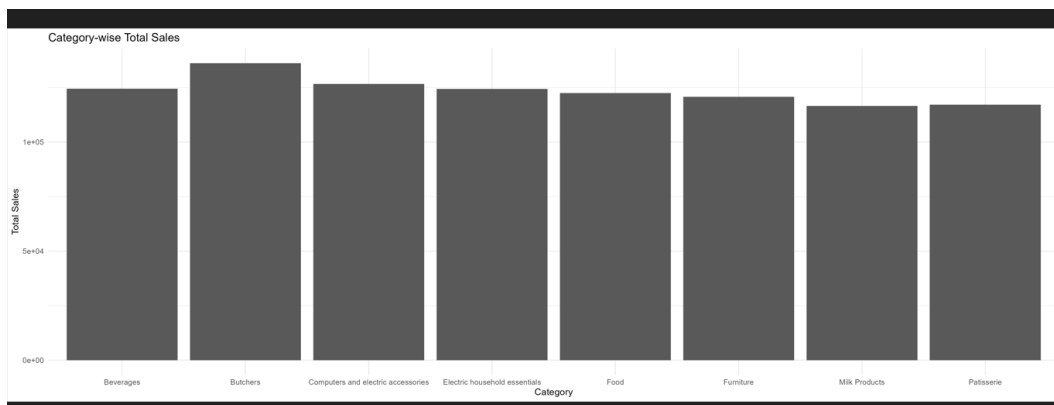
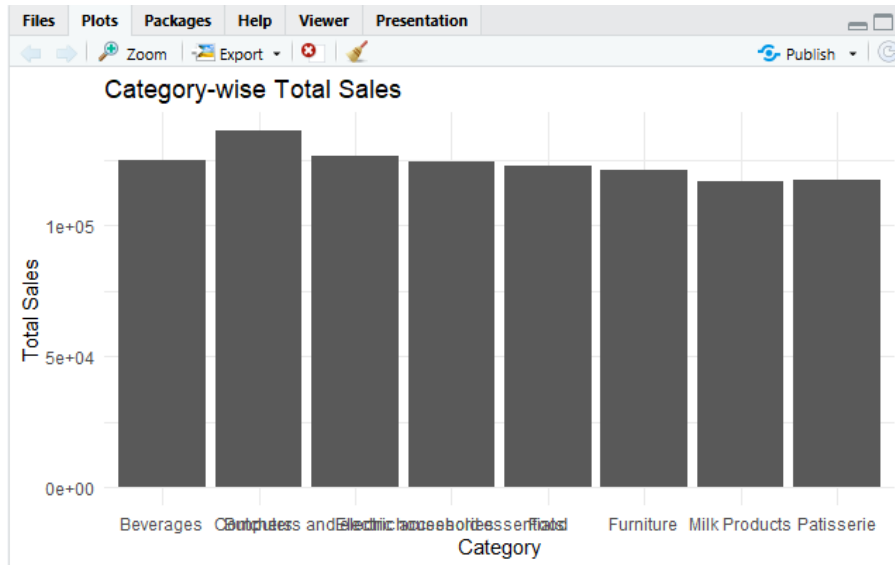
7.1 Bar Chart (Category vs Total Sales)

```
category_sales <- data %>%  
  group_by(Category) %>%  
  summarise(Total_Sales = sum(Total.Spent))
```



	Category	Total_Sales
1	Beverages	121643.0
2	Butchers	132046.5
3	Computers And Electric Accessories	126211.5
4	Electric Household Essentials	121921.5
5	Food	120794.5
6	Furniture	118296.5
7	Milk Products	115718.5
8	Patisserie	116711.0

```
ggplot(category_sales, aes(x = Category, y = Total_Sales)) +  
  geom_bar(stat = "identity") +  
  labs(  
    title = "Category-wise Total Sales",  
    x = "Category",  
    y = "Total Sales"  
  ) +  
  theme_minimal()  
ggsave("category_sales_bar.png", width = 17, height = 6)
```



Interpretation of Category-wise Total Sales

The bar chart shows total sales for each product category.

From the chart, **Butchers has the highest total sales** among all categories.

☞ **This means:**

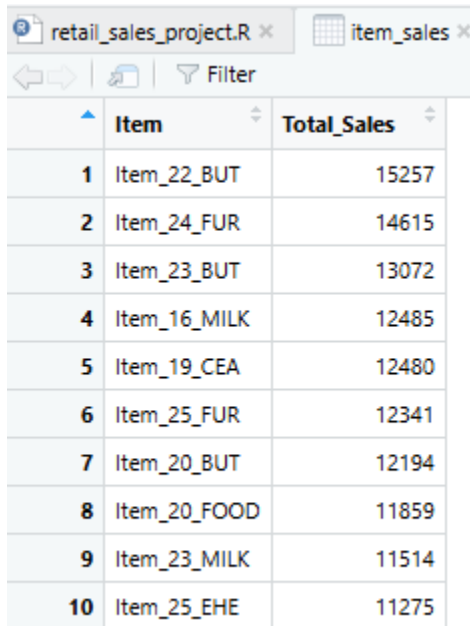
- Butchers products are the **most in demand**
- Customers buy more from this category than others
- It is the **top revenue-generating category** in the store

So, the store should:

- ✓ Focus more on Butchers items
- ✓ Keep strong stock levels
- ✓ Promote this category more

7.2 Bar Chart (Top 10 Items by Sales)

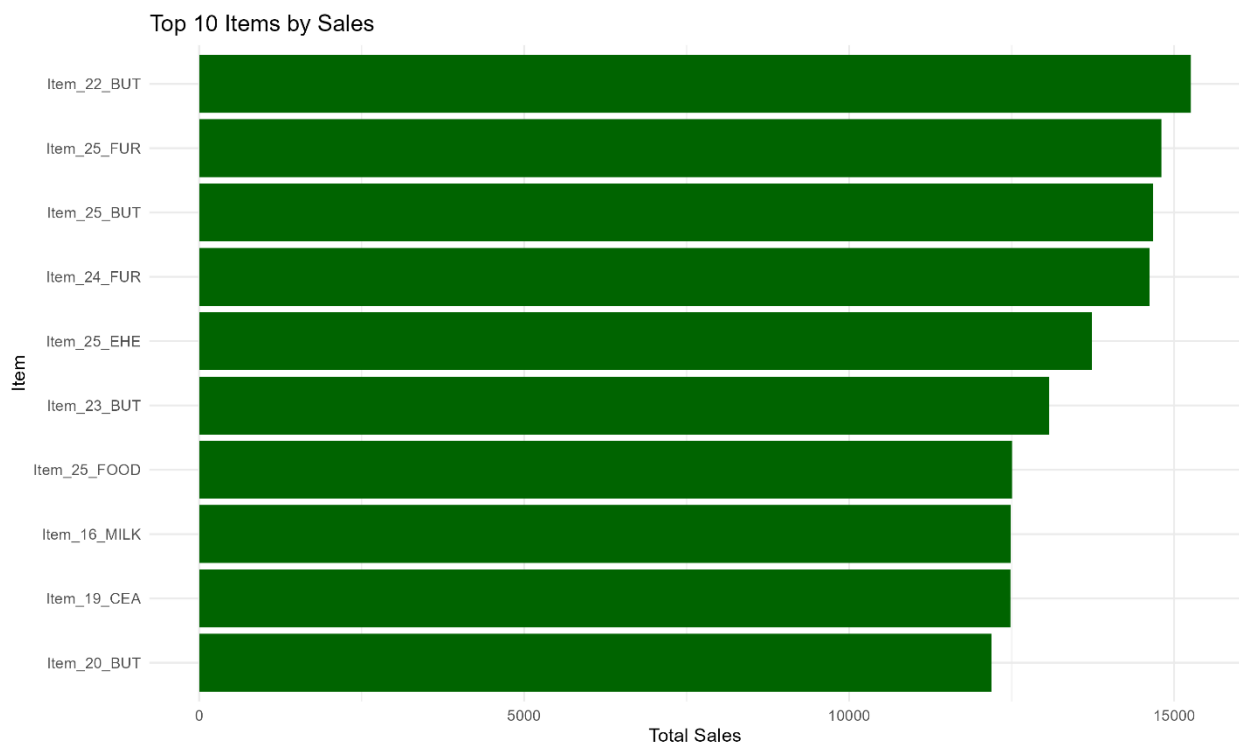
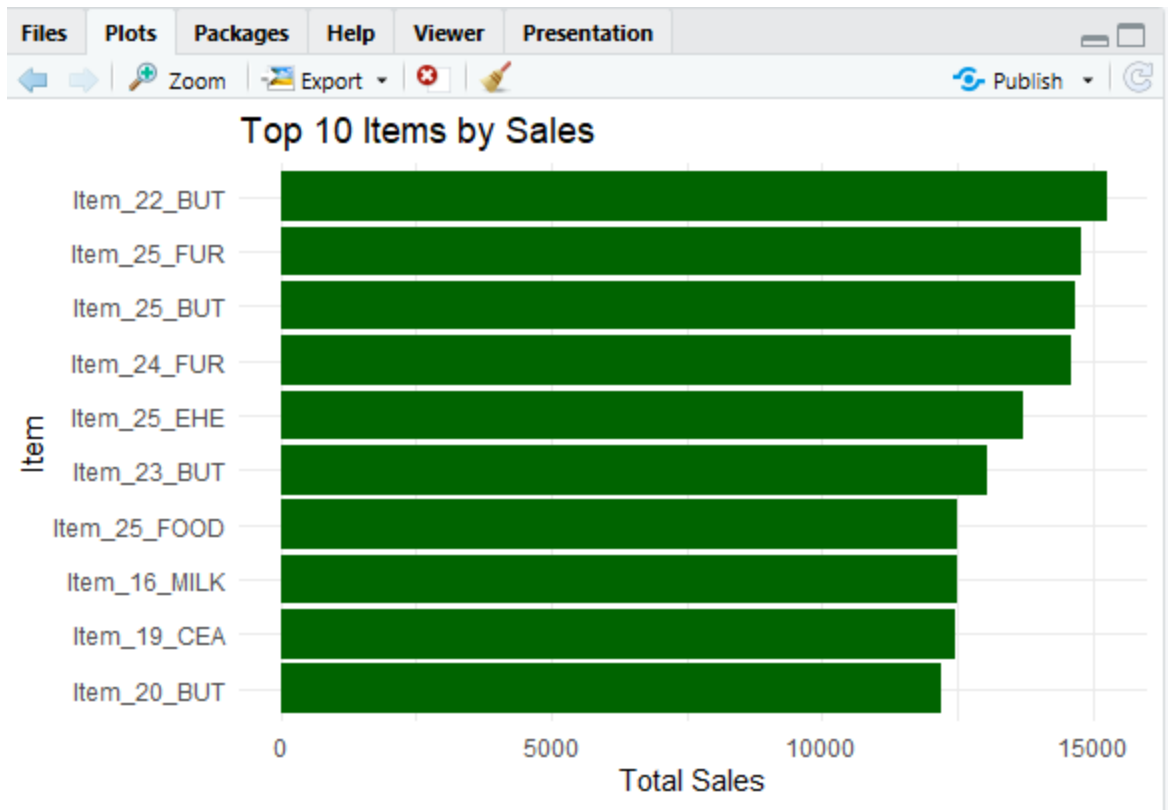
```
item_sales <- data %>%  
  group_by(Item) %>%  
  summarise(Total_Sales = sum(Total.Spent)) %>%  
  arrange(desc(Total_Sales)) %>%  
  head(10) # only top 10 items
```



The screenshot shows a data table with two columns: 'Item' and 'Total_Sales'. The table is sorted in descending order of 'Total_Sales'. The top 10 items are listed, with their corresponding 'Total_Sales' values.

	Item	Total_Sales
1	Item_22_BUT	15257
2	Item_24_FUR	14615
3	Item_23_BUT	13072
4	Item_16_MILK	12485
5	Item_19_CEA	12480
6	Item_25_FUR	12341
7	Item_20_BUT	12194
8	Item_20_FOOD	11859
9	Item_23_MILK	11514
10	Item_25_EHE	11275

```
ggplot(item_sales, aes(x = reorder(Item, Total_Sales), y =  
Total_Sales)) +  
  geom_bar(stat = "identity", fill = "darkgreen") +  
  coord_flip() +  
  labs(title = "Top 10 Items by Sales", x = "Item", y = "Total  
Sales") +  
  theme_minimal()  
  
ggsave("top_item_sales.png", width = 10, height = 6)
```

Interpretation of Top 10 Items by Sales

The bar chart shows the **Top 10 Items by Total Sales**.

On the **Y-axis**, we have the *Item names*, and on the **X-axis**, we have *Total Sales*.

From the chart, **Item 22 BUT has the highest total sales** among all items.

This means Item 22 is the **best-selling product** in the store.

✦ What this tells us:

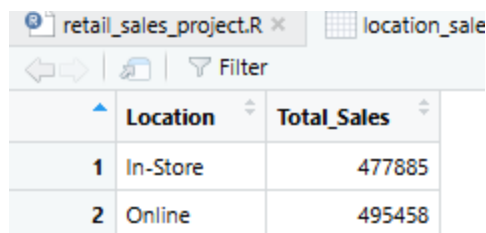
- Customers really like Item 22
- It has the strongest demand
- It is an important product for revenue

☞ The store should:

- Keep Item 22 **well-stocked**
- Promote it more in ads and online
- Use it as a **featured product**

7.3 Bar Chart (Sales by Location)

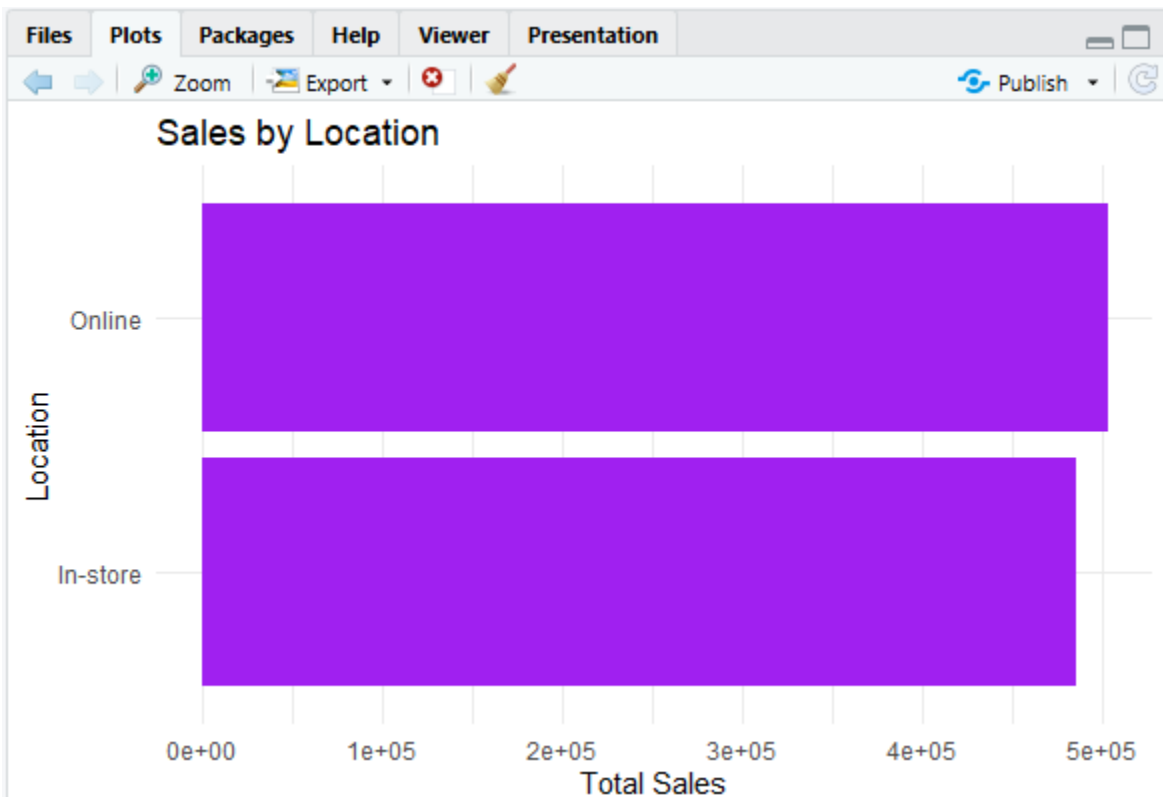
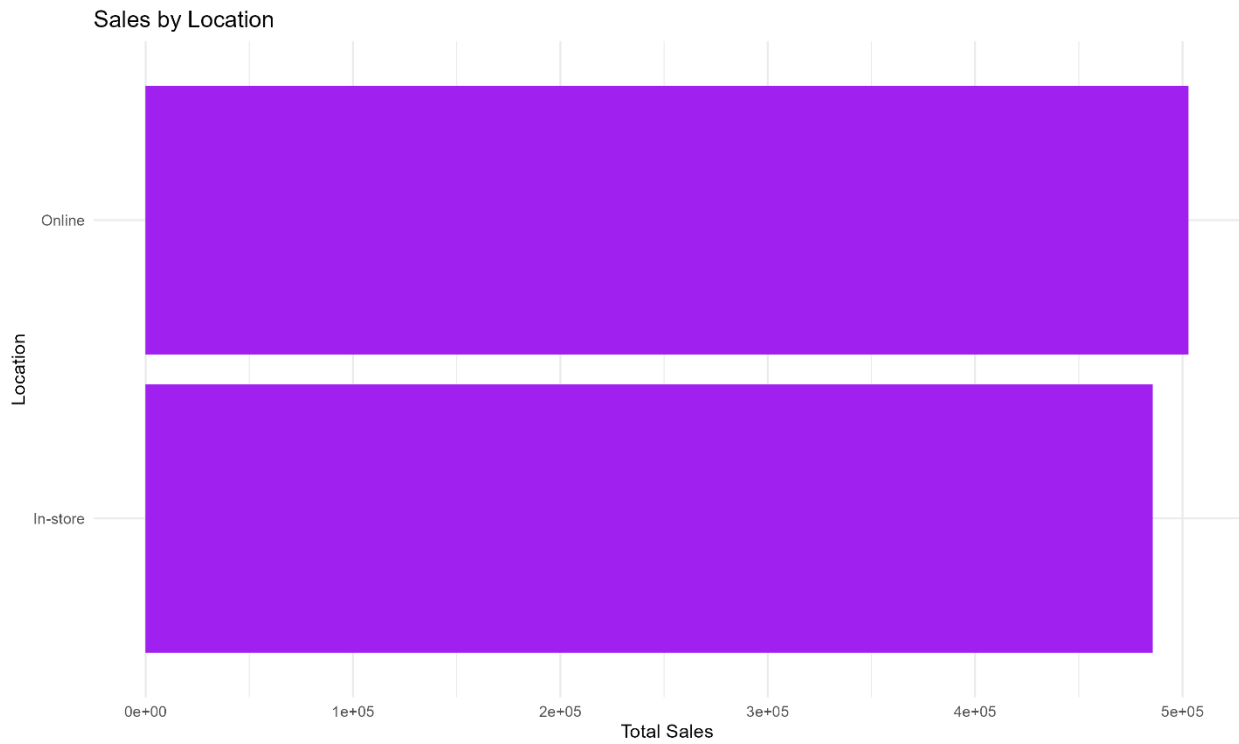
```
location_sales <- data %>%  
  group_by(Location) %>%  
  summarise(Total_Sales = sum(Total.Spent))
```



The screenshot shows an RStudio window with a file named 'retail_sales_project.R' and a data frame 'location_sales'. The data frame has two columns: 'Location' and 'Total_Sales'. The data is as follows:

	Location	Total_Sales
1	In-Store	477885
2	Online	495458

```
ggplot(location_sales, aes(x = reorder(Location, Total_Sales), y  
= Total_Sales)) +  
  geom_bar(stat = "identity", fill = "purple") +  
  coord_flip() +  
  labs(title = "Sales by Location", x = "Location", y = "Total  
Sales") +  
  theme_minimal()  
ggsave("top_item_sales.png", width = 10, height = 6)
```



Interpretation of Sales by Location Chart

The bar chart shows that **Online sales are the highest** compared to other locations. This means most customers prefer buying from our store through the **online platform** rather than visiting physically. This trend suggests that:

- Customers like the **convenience of online shopping**
- Our online presence is strong and effective
- Digital marketing and delivery services are working well

So, focusing more on **online promotions, social media, and website/app improvements** can help increase sales even further.

7.4 Scatter plot (Price.Per.Unit by Quantity)

```
ggplot(data, aes(x = Price.Per.Unit, y = Quantity, color =  
Category)) +  
  
  geom_point(alpha = 0.7) +  
  
  labs(title = "Price vs Quantity by Category", x = "Price per  
Unit", y = "Quantity Sold") +  
  
  theme_minimal()  
  
ggsave("sales_by_location.png", width = 10, height = 6)
```



What does the graph show?

- **X-axis:** Price per Unit
- **Y-axis:** Quantity Sold
- Each color represents a **product category** (Beverages, Food, Furniture, etc.)

★ Easy Findings

- **Higher price does not always mean lower sales**
There is no clear pattern that shows “higher price = lower quantity sold.”
Some expensive items are still selling in good quantities.
- **Most sales are in the middle range (4–7 units)**
Most of the dots are between **Quantity 4 and 7**, meaning many products sell a medium amount.
- **All categories sell at many different prices**
Every category appears across the whole price range.
This means each type of product sells at both low and high prices.
- **Prices range from about 5 to 40**
Both cheap and expensive products are being sold.

7.5 Line Graph (Category vs Total Spent)

```
#creating line chart

# Load dataset

gemonline <- read.csv("retail_store_sales_cleaned.csv")

# Summarize total spent by category

category_summary <- aggregate(Total.Spent ~ Category, data =
gemonline, sum)

# Create line chart

ggplot(category_summary, aes(x = Category, y = Total.Spent,
group = 1)) +

  geom_line(color = "blue") +

  geom_point(color = "red", size = 3) +

  labs(

    title = "Total Spent by Category",

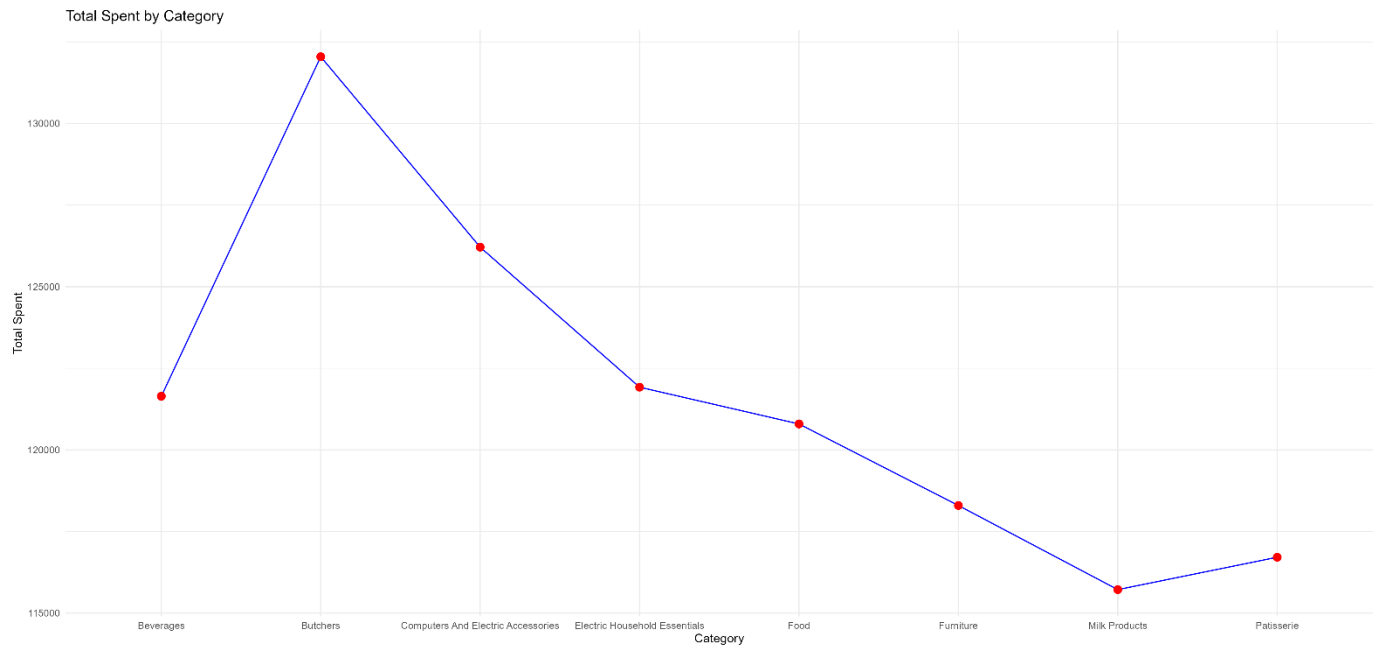
    x = "Category",

    y = "Total Spent"

  ) +

  theme_minimal()

ggsave("Line_graph.png", width=17, height=8)
```



Interpretation of Line graph

- The line graph shows the total sales for each product category. It is clearly observed that the *Butchers* category has the highest total spending among all categories.
- This indicates that customers spend more money on meat and related products more frequently compared to other categories.
- The strong sales of the Butchers category suggests a high and consistent demand for fresh food items, making it a key contributor to overall store revenue.
- Other categories show comparatively lower sales, which highlights the importance of focusing marketing and inventory efforts on high-performing categories like Butchers.

8. Clean data (tidying strings)

8.1 Tidying Strings by `str_trim()` |> `str_to_title()`

We notice in our data that columns like `Category` or `Payment.Method` might have extra spaces, inconsistent capitalization, etc.

```
# Clean strings

retail <- read.csv("retail_store_sales_cleaned.csv")

retail$Category <- retail$Category |> str_trim() |> str_to_title()
retail$Payment.Method <- retail$Payment.Method |> str_trim() |>
str_to_title()
retail$Location <- retail$Location |> str_trim() |> str_to_title()
```

Why:

- `str_trim()` removes extra spaces at start/end.
- `str_to_title()` makes all text consistent (e.g., "digital wallet" → "Digital Wallet").

Benefit: Prevents duplicate categories in analysis (like “digital wallet” vs “Digital Wallet”).

	A	B	C	D	E	F	G	H	I	J	K
1	Transaction.ID	Customer.ID	Category	Item	Price.Per.Unit	Quantity	Total.Spent	Payment.Method	Location	Transaction.Date	Discount.Applied
2	TXN_6867343	CUST_09	Patisserie	Item_10_PAT	18.5	10	185	Digital Wallet	Online	8/4/2024	TRUE
3	TXN_3731986	CUST_22	Milk Products	Item_17_MILK	29	9	261	Digital Wallet	Online	23/07/2023	TRUE
4	TXN_9303719	CUST_02	Butchers	Item_12_BUT	21.5	2	43	Credit Card	Online	5/10/2022	FALSE
5	TXN_4575373	CUST_05	Food	Item_6_FOOD	12.5	7	87.5	Digital Wallet	Online	2/10/2022	FALSE
6	TXN_3652209	CUST_07	Food	Item_1_FOOD	5	8	40	Credit Card	In-Store	10/6/2023	TRUE
7	TXN_9728486	CUST_23	Furniture	Item_16_FUR	27.5	1	27.5	Credit Card	In-Store	26/04/2023	FALSE
8	TXN_2722661	CUST_25	Butchers	Item_22_BUT	36.5	3	109.5	Cash	Online	14/03/2024	FALSE
9	TXN_8776416	CUST_22	Butchers	Item_3_BUT	8	9	72	Cash	In-Store	14/12/2024	TRUE
10	TXN_5874772	CUST_23	Food	Item_2_FOOD	6.5	7	45.5	Cash	Online	9/9/2023	TRUE
11	TXN_4413070	CUST_14	Patisserie	Item_24_PAT	39.5	6	237	Digital Wallet	In-Store	20/05/2022	FALSE
12	TXN_7563311	CUST_23	Patisserie	Item_17_PAT	29	8	232	Cash	Online	16/11/2024	TRUE
13	TXN_4396807	CUST_17	Electric Household Essentials	Item_13_EHE	23	1	23	Digital Wallet	In-Store	7/2/2022	FALSE
14	TXN_3314099	CUST_04	Food	Item_10_FOOD	18.5	1	18.5	Cash	Online	30/09/2023	FALSE
15	TXN_9980736	CUST_13	Furniture	Item_14_FUR	24.5	2	49	Cash	Online	29/08/2023	FALSE
16	TXN_9057609	CUST_18	Butchers	Item_20_BUT	33.5	4	134	Credit Card	In-Store	16/03/2022	TRUE
17	TXN_1599706	CUST_14	Furniture	Item_25_FUR	41	10	410	Credit Card	In-Store	24/03/2024	TRUE
18	TXN_1598860	CUST_08	Food	Item_14_FOOD	24.5	10	245	Cash	In-Store	23/10/2022	TRUE
19	TXN_7138501	CUST_18	Patisserie	Item_22_PAT	36.5	5	182.5	Credit Card	In-Store	10/3/2022	FALSE
20	TXN_5444504	CUST_05	Food	Item_11_FOOD	20	10	200	Digital Wallet	Online	26/04/2022	FALSE
21	TXN_4353295	CUST_22	Patisserie	Item_6_PAT	12.5	8	100	Credit Card	Online	5/4/2022	TRUE
22	TXN_6398436	CUST_15	Beverages	Item_25_BEV	41	7	287	Credit Card	In-Store	3/6/2024	TRUE
23	TXN_9065245	CUST_02	Food	Item_23_FOOD	38	2	76	Digital Wallet	In-Store	10/3/2024	FALSE
24	TXN_6663430	CUST_11	Furniture	Item_10_FUR	18.5	1	18.5	Credit Card	Online	23/04/2024	TRUE

9. Filtering Data

9.1 Filter or subset data (dplyr)

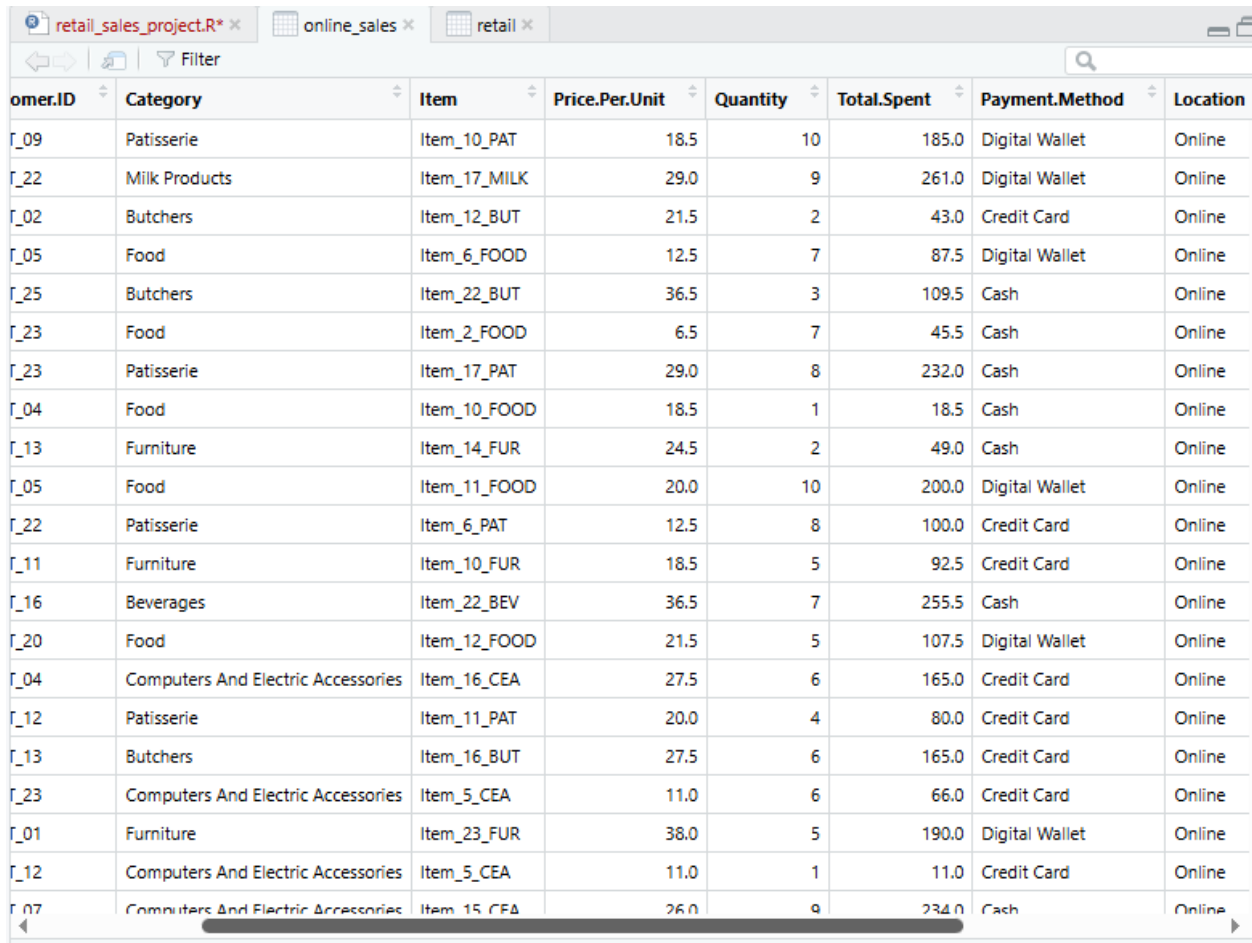
Suppose we want to analyze **only online transactions**:

```
online_sales <- retail |>
filter(Location == "Online")
```

Why:

- `filter()` helps us focus on specific rows.

Benefit: We can analyze online sales separately from in-store sales for better business decisions.



Order.ID	Category	Item	Price.Per.Unit	Quantity	Total.Spent	Payment.Method	Location
109	Patisserie	Item_10_PAT	18.5	10	185.0	Digital Wallet	Online
122	Milk Products	Item_17_MILK	29.0	9	261.0	Digital Wallet	Online
102	Butchers	Item_12_BUT	21.5	2	43.0	Credit Card	Online
105	Food	Item_6_FOOD	12.5	7	87.5	Digital Wallet	Online
125	Butchers	Item_22_BUT	36.5	3	109.5	Cash	Online
123	Food	Item_2_FOOD	6.5	7	45.5	Cash	Online
123	Patisserie	Item_17_PAT	29.0	8	232.0	Cash	Online
104	Food	Item_10_FOOD	18.5	1	18.5	Cash	Online
113	Furniture	Item_14_FUR	24.5	2	49.0	Cash	Online
105	Food	Item_11_FOOD	20.0	10	200.0	Digital Wallet	Online
122	Patisserie	Item_6_PAT	12.5	8	100.0	Credit Card	Online
111	Furniture	Item_10_FUR	18.5	5	92.5	Credit Card	Online
116	Beverages	Item_22_BEV	36.5	7	255.5	Cash	Online
120	Food	Item_12_FOOD	21.5	5	107.5	Digital Wallet	Online
104	Computers And Electric Accessories	Item_16_CEA	27.5	6	165.0	Credit Card	Online
112	Patisserie	Item_11_PAT	20.0	4	80.0	Credit Card	Online
113	Butchers	Item_16_BUT	27.5	6	165.0	Credit Card	Online
123	Computers And Electric Accessories	Item_5_CEA	11.0	6	66.0	Credit Card	Online
101	Furniture	Item_23_FUR	38.0	5	190.0	Digital Wallet	Online
112	Computers And Electric Accessories	Item_5_CEA	11.0	1	11.0	Credit Card	Online
107	Computers And Electric Accessories	Item_15_CEA	26.0	9	234.0	Cash	Online

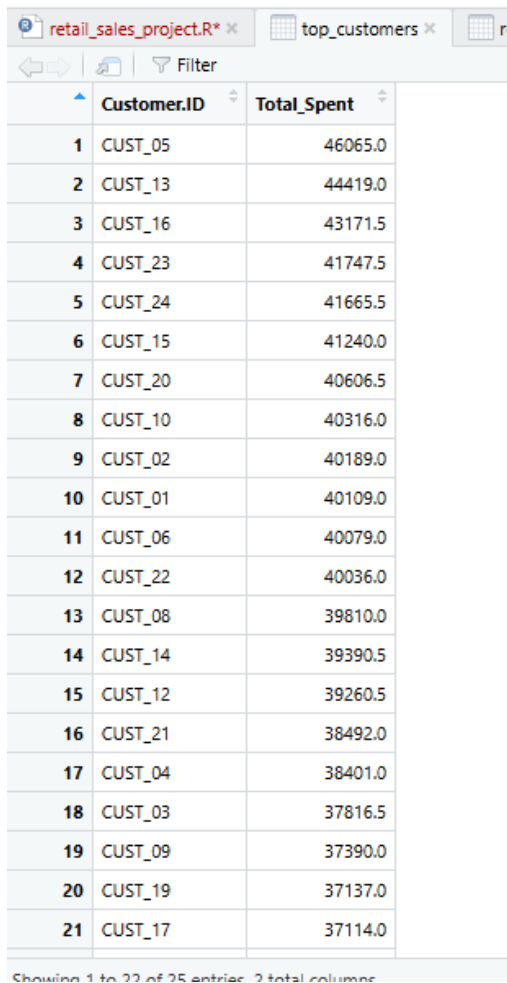
10. Grouping and Summarizing Data

10.1 Find top customer spending

```
top_customers <- retail |>
  group_by(Customer.ID) |>
  summarize(Total_Spent = sum(Total.Spent)) |>
  arrange(desc(Total_Spent))
```

Why: Find the customers who spend the most.

Benefit: Useful for loyalty programs or targeted marketing.



The screenshot shows an RStudio window with two tabs: 'retail_sales_project.R*' and 'top_customers'. The 'top_customers' tab is active, displaying a data table with two columns: 'Customer.ID' and 'Total_Spent'. The table lists 21 customers, sorted by their total spending in descending order. The first customer, CUST_05, has the highest spending at 46065.0, while the 21st customer, CUST_17, has the lowest spending at 37114.0. The status bar at the bottom indicates 'Showing 1 to 21 of 25 entries. 2 total columns'.

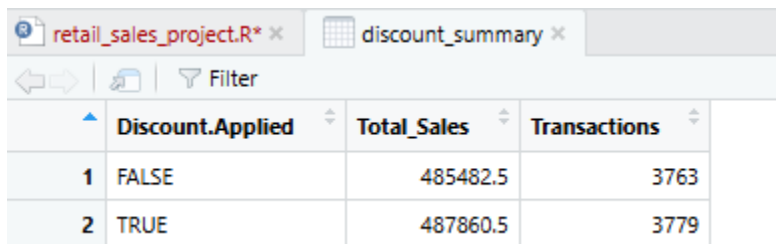
	Customer.ID	Total_Spent
1	CUST_05	46065.0
2	CUST_13	44419.0
3	CUST_16	43171.5
4	CUST_23	41747.5
5	CUST_24	41665.5
6	CUST_15	41240.0
7	CUST_20	40606.5
8	CUST_10	40316.0
9	CUST_02	40189.0
10	CUST_01	40109.0
11	CUST_06	40079.0
12	CUST_22	40036.0
13	CUST_08	39810.0
14	CUST_14	39390.5
15	CUST_12	39260.5
16	CUST_21	38492.0
17	CUST_04	38401.0
18	CUST_03	37816.5
19	CUST_09	37390.0
20	CUST_19	37137.0
21	CUST_17	37114.0

10.2 Check discounts applied

```
discount_summary <- retail |>
  group_by(Discount.Applied) |>
  summarize(Total_Sales = sum(Total.Spent), Transactions = n())
```

Why: See how discounts affect sales.

Benefit: Helps decide **if discounts boost sales effectively**.



The screenshot shows an RStudio window with two tabs: 'retail_sales_project.R*' and 'discount_summary'. The 'discount_summary' tab is active, displaying a data table with three columns: 'Discount.Applied', 'Total_Sales', and 'Transactions'. The table has two rows of data. The first row shows 'Discount.Applied' as FALSE, 'Total_Sales' as 485482.5, and 'Transactions' as 3763. The second row shows 'Discount.Applied' as TRUE, 'Total_Sales' as 487860.5, and 'Transactions' as 3779. The interface includes navigation arrows and a 'Filter' button above the table.

	Discount.Applied	Total_Sales	Transactions
1	FALSE	485482.5	3763
2	TRUE	487860.5	3779

11. Removing Duplicate transactions

11.1 Clean duplicates (distinct transactions)

```
unique_transactions <- retail |>
distinct(Customer.ID, Item, .keep_all = TRUE)

# Save unique transactions to a CSV
write.csv(unique_transactions, "unique_transactions.csv",
row.names = FALSE)
```

What does this code do?

```
unique_transactions <- retail |>
distinct(Customer.ID, Item, .keep_all = TRUE)
```

- `retail` → Your original dataset with all transactions.
- `distinct(Customer.ID, Item, .keep_all = TRUE)` → This tells R:
 1. Look at **Customer.ID** and **Item** columns.
 2. If the **same customer bought the same item multiple times**, keep **only the first occurrence**.
 3. `.keep_all = TRUE` → Keep **all other columns** (like Price, Quantity, Total.Spent, etc.) for that first occurrence.

So after this step:

- Each **customer-item pair appears only once**.
- Customers themselves can still appear multiple times if they bought **different items**.
- Transactions are “unique” in the sense that we’re only counting **the first purchase of each item per customer**, ignoring repeated buys of the same item.

In retail data analysis, we often want to:

1. **Avoid double-counting items**
 - If we are calculating “how many customers bought Item_10_PAT,” we don’t want one customer buying it 3 times to count as 3 different customers.
2. **Understand customer preferences**
 - Unique transactions help us see **which items each customer has bought at least once**.
3. **Simplify data**
 - Raw data can have **thousands of repeated transactions**, which makes analysis messy. Cleaning duplicates creates a **smaller, easier-to-analyze dataset**.

- Customers can appear **multiple times**, but each row is a **unique combination of customer and item**.
- Example:

Customer.ID	Item	Total.Spent
CUST_01	Item_10_PAT	100
CUST_01	Item_12_BUT	50
CUST_01	Item_10_PAT	120

After `distinct(Customer.ID, Item, .keep_all = TRUE)` →

Customer.ID	Item	Total.Spent
CUST_01	Item_10_PAT	100
CUST_01	Item_12_BUT	50

- Notice the **second purchase of Item_10_PAT is removed**, but the customer still appears for other items.

The screenshot shows an RStudio window with a project named 'retail_sales_project.R'. It displays a data table with 11 transactions. The columns are Transaction.ID, Customer.ID, Category, Item, Price.Per.Unit, Quantity, Total.Spent, and Pay. The data is as follows:

	Transaction.ID	Customer.ID	Category	Item	Price.Per.Unit	Quantity	Total.Spent	Pay
1	TXN_6867343	CUST_09	Patisserie	Item_10_PAT	18.5	10	185.0	
2	TXN_3731986	CUST_22	Milk Products	Item_17_MILK	29.0	9	261.0	
3	TXN_9303719	CUST_02	Butchers	Item_12_BUT	21.5	2	43.0	
4	TXN_4575373	CUST_05	Food	Item_6_FOOD	12.5	7	87.5	
5	TXN_3652209	CUST_07	Food	Item_1_FOOD	5.0	8	40.0	
6	TXN_9728486	CUST_23	Furniture	Item_16_FUR	27.5	1	27.5	
7	TXN_2722661	CUST_25	Butchers	Item_22_BUT	36.5	3	109.5	
8	TXN_8776416	CUST_22	Butchers	Item_3_BUT	8.0	9	72.0	
9	TXN_5874772	CUST_23	Food	Item_2_FOOD	6.5	7	45.5	
10	TXN_4413070	CUST_14	Patisserie	Item_24_PAT	39.5	6	237.0	
11	TXN_7563311	CUST_23	Patisserie	Item_17_PAT	29.0	8	232.0	

Showing 1 to 11 of 3,552 entries, 11 total columns

12. Making a package

STEP 1: Package name

📁 retailStorePkg

STEP 2: Package folder creation

RStudio Console:

```
dir.create("retailStorePkg")
```

```
setwd("retailStorePkg")
```

STEP 3: Basic structure

```
file.create("DESCRIPTION")  
file.create("NAMESPACE")  
dir.create("R")
```

STEP 4: DESCRIPTION file

Write the following in the description file

```
Package: retailStorePkg  
Title: Retail Store Sales Analysis Package  
Description: Retail store sales data analysis functions.  
Version: 1.0  
Author: Aamna Gull  
License: MIT
```

STEP 5: Function 1 – Total Sales

Create a file inside 📁 R folder

```
file.create("R/total_sales.R")
```

Code:

```
total_sales <- function(data) {  
  sum(data$Total.Spent, na.rm = TRUE)  
}
```

☞ This Function will calculate the total sales of the store.

STEP 6: Function 2 – Category-wise Sales

```
file.create("R/category_sales.R")
```

Code:

```
category_sales <- function(data) {  
  aggregate(Total.Spent ~ Category, data, sum)  
}
```

☞ This function will tell which category has the highest sales.

STEP 7: Function 3 – Top Selling Items

```
file.create("R/top_items.R")
```

Code:

```
top_items <- function(data, n = 5) {  
  sales <- aggregate(Quantity ~ Item, data, sum)  
  sales[order(-sales$Quantity), ][1:n, ]  
}
```

☞ Top selling items show

STEP 8: Function 4 – Average Order Value

```
file.create("R/average_order_value.R")
```

```
average_order_value <- function(data) {  
  mean(data$Total.Spent, na.rm = TRUE)  
}
```

☞ It will show the average order value of store

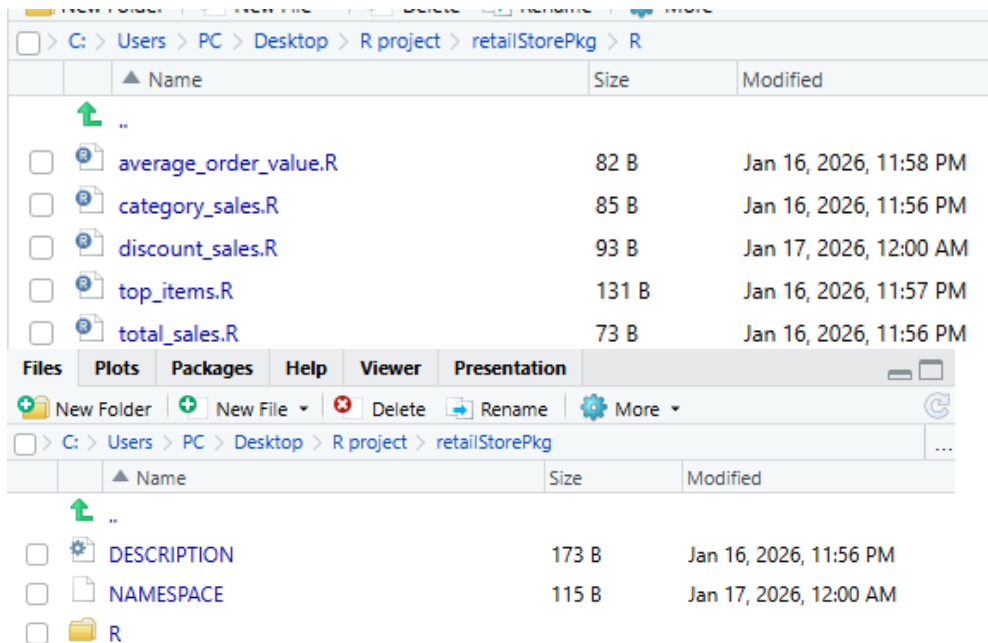
STEP 9: Function 5 – Discount Analysis

```
file.create("R/discount_sales.R")
```

```
discount_sales <- function(data) {  
  aggregate(Total.Spent ~ Discount.Applied, data, sum)}
```

☞ It will show the total sales when discount is applied and total sales when discount is not applied

```
> dir.create("retailStorePkg")  
> setwd("retailStorePkg")  
> file.create("DESCRIPTION")  
[1] TRUE  
> file.create("NAMESPACE")  
[1] TRUE  
> dir.create("R")  
> file.create("R/total_sales.R")  
[1] TRUE  
> file.create("R/category_sales.R")  
[1] TRUE  
> file.create("R/top_items.R")  
[1] TRUE  
> file.create("R/average_order_value.R")  
[1] TRUE  
> file.create("R/discount_sales.R")  
[1] TRUE
```



STEP 10: NAMESPACE file update

Type the following in the namespace

```
export(total_sales)
export(category_sales)
export(top_items)
export(average_order_value)
export(discount_sales)
```

STEP 11: Package load

```
install.packages("devtools")
library(devtools)
load_all()
```

```
> install.packages("devtools")
```

WARNING: Rtools is required to build R packages but is not currently installed. Please download and install the appropriate version of Rtools before proceeding:

<https://cran.rstudio.com/bin/windows/Rtools/>

Installing package into 'C:/Users/PC/AppData/Local/R/win-library/4.5'
(as 'lib' is unspecified)

also installing the dependencies 'Rcpp', 'credentials', 'zip', 'gitcreds', 'ini', 'httpuv', 'later', 'otel', 'promises', 'sourcetools', 'xtable', 'fansi', 'diffobj', 'gert', 'gh', 'projroot', 'whisker', 'shiny', 'downlit', 'httr2', 'htmlwidgets', 'xopen', 'brew', 'commonmark', 'brio', 'praise', 'waldo', 'usethis', 'desc', 'ellipsis', 'miniUI', 'pkgbuild', 'pkgdown', 'pkgload', 'profvis', 'rcmdcheck', 'remotes', 'roxygen2', 'rversions', 'sessioninfo', 'testthat', 'urlchecker'

```
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/Rcpp_1.1.1.zip'
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/credentials_2.0.3.zip'
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/zip_2.3.3.zip'
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/gitcreds_0.1.2.zip'
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/ini_0.3.1.zip'
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/httpuv_1.6.16.zip'
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/later_1.4.5.zip'
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/otel_0.2.0.zip'
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/promises_1.5.0.zip'
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/sourcetools_0.1.7-1.zip'
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/xtable_1.8-4.zip'
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/fansi_1.0.7.zip'
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/diffobj_0.3.6.zip'
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/gert_2.3.1.zip'
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/gh_1.5.0.zip'
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/projroot_2.1.1.zip'
```

```
> library(devtools)
```

Loading required package: usethis

```
> load_all()
```

i Loading retailStorePkg

There were 50 or more warnings (use `warnings()` to see the first 50)

STEP 12: Using in my project

CSV read

```
data <- read.csv("retail_store_sales_cleaned.csv")
```

Functions use

```
total_sales(data)
category_sales(data)
top_items(data, 5)
average_order_value(data)
discount_sales(data)
```

```
> setwd("C:/Users/PC/Desktop/R project")

> data <- read.csv("retail_store_sales_cleaned.csv")
> total_sales(data)
[1] 973343
> category_sales(data)
      Category Total. Spent
1      Beverages  121643.0
2      Butchers  132046.5
3 Computers And Electric Accessories  126211.5
4      Electric Household Essentials  121921.5
5              Food  120794.5
6      Furniture  118296.5
7      Milk Products  115718.5
8      Patisserie  116711.0
> top_items(data, 5)
      Item Quantity
63 Item_16_MILK    454
23 Item_11_MILK    440
114 Item_22_BUT    418
27 Item_12_CEA     403
89 Item_2_BEV      403
> average_order_value(data)
[1] 129.0564
> discount_sales(data)
Discount.Applied Total. Spent
1      FALSE    485482.5
2       TRUE    487860.5
> |
```

13. Menu designing

```
#MENU DESIGN

# CSV read

data <- read.csv("retail_store_sales_cleaned.csv")

menu_column <- "Category"

options <- unique(na.omit(data[[menu_column]]))

menu <- paste0(seq_along(options), ". ", options)

cat("Select a Category:\n")

cat(menu, sep = "\n")


choice <- as.integer(readline("Enter your choice number: "))


# initialize variables (IMPORTANT)

selected_value <- NULL

result <- NULL


if (is.na(choice) || choice < 1 || choice > length(options)) {
  cat("Invalid choice")
} else {
  selected_value <- options[choice]
  result <- subset(data, data[[menu_column]] == selected_value)
  print(result)
}
```

```

Console Terminal × Background Jobs ×
R 4.5.2 · C:/Users/PC/Desktop/R project/
> #MENU DESIGN
> # CSV read
> data <- read.csv("retail_store_sales_cleaned.csv")
> menu_column <- "Category"
> options <- unique(na.omit(data[[menu_column]]))
> menu <- paste0(seq_along(options), ". ", options)
> cat("select a category:\n")
Select a Category:
> cat(menu, sep = "\n")
1. Patisserie
2. Milk Products
3. Butchers
4. Food
5. Furniture
6. Electric Household Essentials
7. Beverages
8. Computers And Electric Accessories
> choice <- as.integer(readline("Enter your choice number: "))
Enter your choice number: 2
> # initialize variables (IMPORTANT)
> selected_value <- NULL
> result <- NULL
> if (is.na(choice) || choice < 1 || choice > length(options)) {
+   cat("Invalid choice")
+ } else {
+   selected_value <- options[choice]
+   result <- subset(data, data[[menu_column]] == selected_value)
+   print(result)
+ }
  Transaction.ID Customer.ID Category Item Price.Per.Unit Quantity
2      TXN_3731986   CUST_22 Milk Products Item_17_MILK      29.0      9
35     TXN_3424571   CUST_09 Milk Products Item_6_MILK      12.5      7
37     TXN_1998771   CUST_25 Milk Products Item_16_MILK      27.5      3
57     TXN_5705131   CUST_19 Milk Products Item_4_MILK       9.5      4
79     TXN_6989569   CUST_11 Milk Products Item_21_MILK     35.0      2

```

Transaction.ID	Customer.ID	Category	Item	Price.Per.Unit	Quantity	Total.Spent	Payment.Method	
2	TXN_3731986	CUST_22	Milk Products	Item_17_MILK	29.0	9	261.0	Digital Wallet
35	TXN_3424571	CUST_09	Milk Products	Item_6_MILK	12.5	7	87.5	Credit Card
37	TXN_1998771	CUST_25	Milk Products	Item_16_MILK	27.5	3	82.5	Digital Wallet
57	TXN_5705131	CUST_19	Milk Products	Item_4_MILK	9.5	4	38.0	Digital Wallet
79	TXN_6989569	CUST_11	Milk Products	Item_21_MILK	35.0	2	70.0	Credit Card
82	TXN_3831408	CUST_18	Milk Products	Item_1_MILK	5.0	10	50.0	Digital Wallet
85	TXN_5111184	CUST_19	Milk Products	Item_11_MILK	20.0	6	120.0	Credit Card
104	TXN_4448047	CUST_03	Milk Products	Item_8_MILK	15.5	6	93.0	Credit Card
107	TXN_8959624	CUST_09	Milk Products	Item_1_MILK	5.0	7	35.0	Digital Wallet
109	TXN_8136588	CUST_07	Milk Products	Item_16_MILK	27.5	7	192.5	Digital Wallet
116	TXN_2755139	CUST_21	Milk Products	Item_4_MILK	9.5	1	9.5	Digital Wallet

14. Pivot wider

```
#USING PIVOT WIDER FUNCTION

data <- read.csv("retail_store_sales_cleaned.csv")

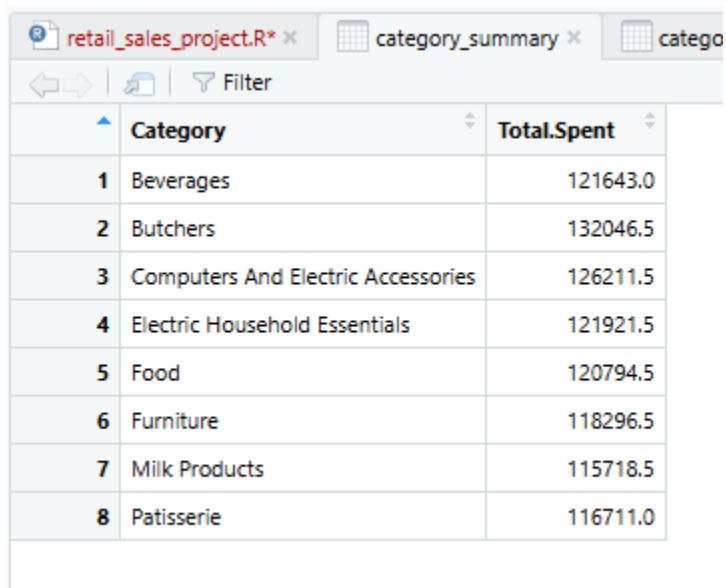
category_summary <- aggregate(

  Total.Spent ~ Category,

  data,

  sum

)
```



The screenshot shows an RStudio window with three tabs: 'retail_sales_project.R*', 'category_summary', and 'category'. The 'category_summary' tab is active, displaying a data table with 8 rows and 2 columns. The columns are 'Category' and 'Total.Spent'. The rows are numbered 1 through 8. The data is as follows:

	Category	Total.Spent
1	Beverages	121643.0
2	Butchers	132046.5
3	Computers And Electric Accessories	126211.5
4	Electric Household Essentials	121921.5
5	Food	120794.5
6	Furniture	118296.5
7	Milk Products	115718.5
8	Patisserie	116711.0

```
category_wide <- category_summary %>%

  pivot_wider(

    names_from = Category,

    values_from = Total.Spent

  )

print(category_wide)
```

retail_sales_project.R* x		category_summary x		category_wide x			
Filter							
	Beverages	Butchers	Computers And Electric Accessories	Electric Household Essentials	Food	Furniture	Milk Produ
1	121643	132046.5	126211.5	121921.5	120794.5	118296.5	115718.5

Environment	History	Connections	Tutorial	
📁 📄 📊 Import Dataset 🔄 282 MiB 🧹				
R Global Environment				
Data				
category_summary		8 obs. of 2 variables		
category_wide		1 obs. of 8 variables		
data		7542 obs. of 11 variables		

15. Conclusion

This project successfully demonstrated the power and versatility of **R Programming** in analyzing retail sales data. Through systematic data cleaning, preprocessing, and visualization, we were able to transform raw data into meaningful insights. The analysis provided a clear understanding of sales trends, customer preferences, and product performance, highlighting areas of strength as well as opportunities for improvement.

By exploring patterns in customer behavior and sales across different categories, the findings can directly support **better inventory management**, ensuring that popular products are always in stock while reducing overstock of less demanded items. Additionally, insights from the analysis can inform **marketing strategies**, allowing the business to design targeted promotions and campaigns that resonate with specific customer segments.

Moreover, the project showcased how R can be used to automate repetitive tasks, perform statistical analysis, and generate visualizations that are easy to interpret for stakeholders. Overall, this work emphasizes the importance of **data-driven decision-making** in retail, demonstrating that a structured analytical approach can significantly enhance operational efficiency, boost sales, and improve overall customer satisfaction.